

JavaScript Callbacks & Higher-Order Functions

What is a Callback?

A callback is a function that is passed as an argument to another function, and is executed later.

Simple Example:

```
function greet(name, callback) {  
  console.log('Hello ' + name);  
  callback(); // Execute the callback  
}  
  
greet('Ahmed', function() {  
  console.log('Nice to meet you!');  
});  
  
// Output:  
// Hello Ahmed  
// Nice to meet you!
```

Real World Example: Validation

```
function validateUser(email, onSuccess, onError) {  
  if (email.includes('@')) {  
    onSuccess('Valid email');  
  } else {  
    onError('Invalid email');  
  }  
}  
  
validateUser('ahmed@gmail.com',  
  (msg) => console.log(msg),  
  (error) => console.log(error)  
);
```

Higher-Order Functions

A function that takes a function as parameter or returns a function.

Example: Function that operates on functions

```
function repeat(fn, times) {  
  for (let i = 0; i < times; i++) {  
    fn();  
  }  
}  
  
repeat(() => console.log('Hello'), 3);
```

Callbacks with Array Methods:

// map, filter, reduce all take callbacks!

```
const numbers = [1, 2, 3, 4, 5];  
  
// Callback in map:  
const doubled = numbers.map(n => n * 2);  
  
// Callback in filter:  
const evens = numbers.filter(n => n % 2 === 0);  
  
// Callback in reduce:  
const sum = numbers.reduce((acc, n) => acc + n, 0);
```